

DATA SCIENCE IN CYBERSECURITY

Final Project Report

Paper: Learning to Detect: A Data-Driven Approach for Network Intrusion Detection Using Machine Learning

Student: Sewar Halifa

Table of Contents

1. Summary of the Source
 2. Critical Evaluation
 3. Feature Engineering Analysis
 4. Reproducibility Analysis
 5. Experimental Results
 6. Conclusions
 7. References
-

1. Summary of the Source

1.1 Introduction

Network intrusion detection has become one of the most important challenges in modern cybersecurity. Organizations continuously exchange large volumes of network traffic, making computer networks attractive targets for cyberattacks. Detecting malicious activities quickly and accurately is essential for preventing unauthorized access, protecting sensitive information, and maintaining system availability.

Traditional intrusion detection systems mainly rely on signature-based techniques, which perform well for previously known attacks but often fail to identify new or evolving threats. Machine learning provides an alternative solution by learning patterns directly from historical network traffic and automatically distinguishing between normal and malicious behavior.

The paper *Learning to Detect: A Data-Driven Approach for Network Intrusion Detection Using Machine Learning* proposes a hierarchical machine learning framework that combines classical supervised learning

algorithms with deep learning techniques to improve intrusion detection performance.

1.2 Problem Being Addressed

The paper addresses the problem of **network intrusion detection**, whose objective is to identify malicious network connections while minimizing classification errors.

Specifically, the proposed framework consists of two sequential stages. During the first stage, network traffic is classified as either **Normal** or **Attack**. During the second stage, only the detected attack samples are further classified into four attack categories.

This project focuses exclusively on the **first stage**, namely the binary intrusion detection problem, which is also the main focus of the implemented notebook.

1.3 Why the Problem Is Important

Cyberattacks continue to increase in both frequency and sophistication. Modern organizations generate massive amounts of network traffic every second, making manual monitoring impossible.

An effective intrusion detection system allows security analysts to detect malicious activities before attackers can compromise sensitive systems or steal confidential information.

At the same time, intrusion detection systems must minimize false alarms. Excessive false positives waste valuable resources, while false negatives allow real attacks to remain undetected. Therefore, achieving a balance between accurate detection and low false alarm rates is a fundamental cybersecurity challenge.

1.4 Proposed Solution

The paper proposes a **hierarchical machine learning framework** for intrusion detection.

The first stage performs **binary classification**, separating network traffic into **Normal** and **Attack** classes. Several supervised learning algorithms are evaluated, including Decision Tree, Random Forest, Support Vector

Machine (SVM), AdaBoost, Gradient Boosting, Naive Bayes, and Multi-Layer Perceptron (MLP). In addition, the authors propose an Autoencoder-based anomaly detection model that learns only normal network behavior.

After malicious traffic has been detected, the second stage classifies attacks into four attack categories using a deep neural network.

Since the objective of this project was to reproduce the binary intrusion detection stage, only the first stage of the proposed framework was implemented and evaluated.

1.5 Dataset

The experiments were performed using the **NSL-KDD** dataset, which is an improved version of the original KDD Cup 1999 intrusion detection dataset.

The dataset contains **41 input features** describing various characteristics of network connections, including protocol information, service type, connection statistics, content-based features, and host-based traffic information.

Although the original dataset contains multiple attack categories, all attack classes were merged into a single **Attack** class in order to create a binary classification problem consisting of:

- **Normal**
- **Attack**

This transformation follows the first stage of the methodology proposed in the paper and enables direct comparison with its binary intrusion detection framework.

1.6 Methodology

The implementation follows the general methodology presented in the selected paper while adapting several practical implementation details.

The experimental pipeline consists of the following stages:

1. Loading the NSL-KDD training and testing datasets.
2. Inspecting the data and verifying data quality.

3. Encoding categorical attributes using Label Encoding.
4. Standardizing numerical features using StandardScaler.
5. Transforming the original attack categories into a binary Attack label.
6. Training multiple supervised learning algorithms.
7. Optimizing hyperparameters using GridSearchCV.
8. Training an Autoencoder using only normal training samples.
9. Detecting anomalies based on reconstruction error.
10. Evaluating every model using multiple classification metrics.

The following machine learning models were implemented:

- Decision Tree
- Random Forest
- Linear Support Vector Machine (LinearSVC)
- AdaBoost
- Gradient Boosting
- Multi-Layer Perceptron (MLP)
- Autoencoder

To ensure a fair comparison, all supervised models were trained using the same preprocessing pipeline and evaluated on the same testing dataset. The Autoencoder followed a different training strategy by learning only normal network traffic and identifying attacks through reconstruction error.

2. Critical Evaluation

2.1 Main Claims of the Paper

The primary claim made by the authors is that combining traditional supervised machine learning algorithms with an Autoencoder-based anomaly detection model provides an effective solution for network intrusion detection. The paper argues that using several supervised classifiers together with a deep learning anomaly detection model enables a comprehensive comparison of different learning approaches while improving the ability to distinguish between normal and malicious network traffic.

A second major claim is that the proposed hierarchical framework improves intrusion detection by dividing the task into two stages. The first stage performs binary classification by separating normal traffic from attack traffic, while the second stage further classifies the detected attacks into specific attack categories. According to the authors, this design simplifies the learning task and leads to better overall performance.

The paper also claims that the Autoencoder is particularly effective because it learns only the characteristics of normal network traffic. Instead of directly learning attack patterns, it detects anomalies through reconstruction error, allowing it to identify malicious connections that differ from normal behavior.

These claims are reasonable and are supported by the experimental results presented in the paper. However, some of them rely on assumptions that deserve further discussion. For example, the paper assumes that reconstruction-based anomaly detection is generally superior to supervised learning without thoroughly examining situations in which supervised models may perform equally well or even better. In addition, although the hierarchical framework is conceptually appealing, the paper provides limited analysis of how much each stage individually contributes to the overall improvement.

Overall, the research presents clear and well-motivated claims, but a more detailed analysis of the individual contribution of each design choice would strengthen the argument and provide greater confidence in the proposed methodology.

2.2 Are the Claims Supported by the Evidence?

The experimental results presented in the paper generally support the authors' main claims. The comparison includes several supervised learning algorithms together with an Autoencoder-based anomaly detection model, and the reported evaluation metrics consistently indicate strong performance for the proposed approach. The use of multiple evaluation metrics, rather than relying only on Accuracy, provides stronger evidence that the proposed methodology performs well on the binary intrusion detection task.

The reproduction study conducted in this project reached similar overall conclusions. Although the exact numerical results differ from those

reported in the paper, the Autoencoder achieved the best overall performance among all evaluated models, while the Multi-Layer Perceptron (MLP) was the strongest supervised classifier. This consistency increases confidence that the proposed methodology is effective for binary intrusion detection.

Nevertheless, some of the evidence presented in the paper could be strengthened. The experiments are conducted exclusively on the NSL-KDD dataset, which is a widely used benchmark but does not fully represent modern network environments or recent attack techniques. Evaluating the proposed framework on additional intrusion detection datasets, such as UNSW-NB15 or CIC-IDS2017, would provide stronger evidence that the methodology generalizes beyond a single benchmark.

Another limitation is that the paper mainly reports overall binary classification performance. While this demonstrates the effectiveness of distinguishing between normal and malicious traffic, it does not analyze whether the model performs equally well across different attack families. Some attack categories may be significantly easier to detect than others, and reporting class-specific performance would provide a more comprehensive evaluation of the proposed framework.

Furthermore, although the Autoencoder achieved the best overall results, the comparison with supervised learning models should be interpreted carefully. The Autoencoder is trained only on normal traffic and detects anomalies through reconstruction error, whereas the supervised models are trained using both normal and attack samples. Because the learning objectives differ, the comparison is informative but not entirely equivalent. A discussion of these differences would strengthen the interpretation of the experimental results.

Overall, the evidence presented in the paper supports the main conclusions, but additional experiments, more diverse datasets, and a deeper analysis of attack-specific performance would provide stronger validation of the authors' claims.

2.3 Evaluation Methodology

The evaluation methodology presented in the paper is generally appropriate for the network intrusion detection problem. The authors evaluate several machine learning algorithms using the same dataset and report multiple evaluation metrics, including Accuracy, Precision,

Recall, F1-score, ROC-AUC, and Matthews Correlation Coefficient (MCC). Using multiple complementary metrics is particularly important in cybersecurity because intrusion detection datasets are often imbalanced, and Accuracy alone may provide a misleading impression of model performance.

The preprocessing pipeline is also consistent across the supervised learning models, which contributes to a fair comparison. However, the paper provides only limited details regarding several preprocessing decisions. For example, it does not clearly explain the exact order in which preprocessing operations were performed or whether all preprocessing steps were fitted exclusively on the training data before being applied to the test set. This information is important because improper preprocessing may introduce data leakage, allowing information from the test set to influence the training process and leading to overly optimistic performance estimates. Although there is no direct evidence that data leakage occurred, the paper would benefit from explicitly describing how this risk was avoided.

Another methodological aspect concerns the encoding of categorical features. The paper does not discuss whether alternative encoding methods were considered. If Label Encoding is used for nominal variables such as protocol type or service, it may unintentionally introduce artificial ordinal relationships between categories. Methods such as One-Hot Encoding could preserve the nominal nature of these features more effectively, although they would increase the dimensionality of the dataset. A discussion of this design choice would strengthen the methodological justification.

The comparison between the Autoencoder and the supervised learning models should also be interpreted carefully. The supervised classifiers are trained using labeled examples from both normal and attack classes, whereas the Autoencoder is trained exclusively on normal traffic and identifies anomalies based on reconstruction error. Since these approaches solve different learning problems, the comparison is useful but not perfectly equivalent. Additional discussion of this difference would help readers interpret the reported performance more accurately.

Finally, the paper focuses mainly on predictive performance and does not evaluate practical deployment aspects such as training time, inference speed, computational cost, or memory consumption. These

factors are particularly important for real-world intrusion detection systems that must analyze large volumes of network traffic in real time. Including such measurements would provide a more comprehensive evaluation of the proposed framework.

Overall, the evaluation methodology is well designed and appropriate for the research objective. Nevertheless, providing additional methodological details, discussing alternative preprocessing strategies, and evaluating computational performance would improve the transparency, reproducibility, and practical relevance of the study.

2.4 Possible Weaknesses and Limitations

Although the paper presents a well-structured methodology and achieves strong experimental results, several limitations should be considered when interpreting its conclusions.

The first limitation is the exclusive use of the NSL-KDD dataset. While NSL-KDD is one of the most widely used benchmark datasets for intrusion detection research, it was created to address the limitations of the older KDD Cup 1999 dataset and therefore does not fully represent modern network environments or contemporary cyber threats. Network traffic patterns, protocols, and attack techniques have evolved significantly in recent years. Consequently, strong performance on NSL-KDD does not necessarily guarantee similar performance in real-world operational environments.

A second limitation is the lack of evaluation on multiple datasets. Testing the proposed methodology only on a single benchmark makes it difficult to determine whether the observed performance generalizes to other network environments. Validation on additional public datasets, such as UNSW-NB15 or CIC-IDS2017, would provide stronger evidence for the robustness and generalization capability of the proposed framework.

Another limitation concerns the analysis of attack categories. During the first stage, all attack types are merged into a single Attack class. Although this simplifies the classification task and is appropriate for binary intrusion detection, it also hides potential differences between attack families. Some attacks may be much easier to detect than others, and reporting class-specific detection performance would

provide a more complete understanding of the strengths and weaknesses of the proposed approach.

The paper also provides limited discussion of the computational requirements of the evaluated models. Performance metrics such as Accuracy and Recall are important, but practical deployment also depends on training time, inference speed, memory consumption, and computational cost. These factors are particularly relevant for intrusion detection systems that must analyze large volumes of network traffic in real time. Including such measurements would improve the practical value of the study.

In addition, several implementation details are not described in sufficient depth. For example, the paper provides only limited information about preprocessing decisions, hyperparameter selection, and the procedure used to determine the Autoencoder reconstruction-error threshold. Although these details do not invalidate the proposed methodology, they make exact reproduction of the published results more difficult.

Finally, the paper focuses primarily on overall classification performance and provides relatively limited discussion of model interpretability. In practical cybersecurity applications, understanding why a model classifies a connection as malicious is often important for security analysts. Additional explainability techniques, such as feature importance analysis or explainable AI methods, could increase the practical usefulness of the proposed framework.

Despite these limitations, the paper presents a solid methodology and provides convincing evidence that machine learning techniques can significantly improve binary network intrusion detection. The identified limitations mainly represent opportunities for future research rather than fundamental weaknesses of the proposed approach.

2.5 Are the Conclusions Justified?

Overall, the conclusions presented in the paper are **largely justified** by the experimental evidence. The reported results consistently demonstrate that machine learning techniques, particularly the Autoencoder-based anomaly detection model, achieve strong

performance on the binary intrusion detection task. The reproduction study conducted in this project reached similar overall conclusions, providing additional support for the effectiveness of the proposed methodology.

However, the conclusions should be interpreted with some caution. The experiments are based on a single benchmark dataset, and the paper does not demonstrate that the proposed framework performs equally well on more recent intrusion detection datasets or in real-world operational environments. Consequently, while the reported results are convincing within the scope of the study, they should not be considered sufficient evidence that the methodology will generalize to every cybersecurity scenario.

The claim that the Autoencoder provides the best overall solution is also reasonable but should be interpreted carefully. The Autoencoder is trained using only normal network traffic, whereas the supervised learning models are trained using labeled examples from both normal and attack classes. Since these approaches address the problem using different learning paradigms, the comparison highlights their relative strengths but should not be viewed as definitive proof that one approach is universally superior.

In addition, the paper successfully demonstrates the effectiveness of the binary intrusion detection stage but provides relatively limited discussion regarding practical deployment challenges. Factors such as computational efficiency, scalability, model interpretability, and robustness to evolving attack patterns are important considerations for real-world cybersecurity systems and deserve further investigation.

Despite these limitations, the paper makes a valuable contribution to the field of intrusion detection. The proposed methodology is well designed, the experimental evaluation is comprehensive, and the reproduced results obtained in this project generally support the authors' main findings. Therefore, the conclusions are justified within the scope of the conducted experiments, although additional validation on modern datasets and real-world environments would further strengthen the overall claims.

3. Feature Engineering Analysis

3.1 Introduction

Feature engineering is one of the most important stages in every machine learning pipeline because the quality of the input features directly influences the performance of the trained models. Although the NSL-KDD dataset is already a refined version of the original KDD Cup 1999 dataset, several preprocessing and feature engineering operations are still required before training machine learning algorithms.

In this project, feature engineering consisted of data inspection, categorical feature encoding, feature standardization, binary label transformation, exploratory data analysis, and correlation analysis. These operations improved the numerical representation of the data while ensuring that all evaluated models were trained using the same input features.

3.2 Features Used

The NSL-KDD dataset contains **41 input features** describing different characteristics of network traffic. According to the paper, these features can be divided into three main categories: **basic connection features, traffic features, and content features**.

Basic features describe the properties of individual network connections, including protocol type, service, and connection duration. Traffic features summarize communication patterns observed over multiple connections, while content features describe suspicious activities within the transmitted data.

Using multiple groups of features allows the machine learning models to capture both individual connection behavior and broader network traffic characteristics, improving their ability to distinguish between normal and malicious activities.

3.3 Data Preprocessing and Transformations

Several preprocessing operations were applied before training the machine learning models.

Categorical Feature Encoding

The NSL-KDD dataset contains three categorical attributes:

- protocol_type
- service

- flag

Machine learning algorithms require numerical input; therefore, these categorical variables were converted into numerical values using **Label Encoding**.

Although the paper applies **LabelCount Encoding** for categorical features, this implementation uses Label Encoding because it is directly supported by Scikit-learn and provides a simple and reproducible preprocessing pipeline. Both approaches convert categorical variables into numerical representations, allowing the models to process network traffic data efficiently.

Feature Standardization

Numerical features were standardized using **StandardScaler** before model training.

The standardization process is defined as

$$z = \frac{x - \mu}{\sigma}$$

where

- x is the original feature value,
- μ is the feature mean,
- σ is the feature standard deviation.

Standardization transforms each feature to approximately zero mean and unit variance, preventing variables with larger numerical ranges from dominating the learning process.

This preprocessing step is particularly important for optimization-based models such as **Linear SVM**, **MLP**, and the **Autoencoder**, all of which benefit from normalized input features.

Binary Label Transformation

The original NSL-KDD dataset contains multiple attack categories.

Since this project reproduces only the first stage of the framework proposed in the paper, all attack categories were merged into a single **Attack** class, while normal network traffic remained labeled as **Normal**.

This transformation converts the original multi-class problem into a binary classification problem that directly matches the objective of the implemented models.

3.4 Was Feature Selection Performed?

Explicit feature selection was **not performed** in this project.

All **41 original features** were retained throughout the experiments.

This decision was made for two reasons. First, it maintains consistency with the methodology presented in the paper. Second, using the complete feature set ensures that all evaluated machine learning models receive identical input data, allowing a fair comparison between their performance.

3.5 Redundancy Analysis

During the exploratory data analysis stage, correlation analysis was performed to better understand the relationships between different features.

Highly correlated variables may contain redundant information and increase model complexity without providing additional predictive value.

The paper also identifies redundant features in the NSL-KDD dataset. For example, the **num_outbound_cmds** feature always has the value zero and therefore contributes no useful information for classification. The authors further analyze feature relationships using correlation heatmaps and scatter plots to identify dependencies between variables.

In this implementation, correlation analysis was primarily used to understand the data rather than remove features. Since the evaluated machine learning models are generally robust to moderate feature redundancy, all original features were retained throughout the experiments.

3.6 Mathematical Perspective

From a mathematical perspective, preprocessing improves the numerical properties of the input data and facilitates the optimization process.

Feature standardization ensures that variables contribute equally during model training, reducing optimization bias caused by different numerical scales.

Encoding categorical variables converts symbolic values into numerical representations that can be processed by machine learning algorithms.

Together, these transformations improve numerical stability and allow gradient-based optimization algorithms to converge more efficiently.

3.7 Cybersecurity Perspective

From a cybersecurity perspective, each feature describes a different aspect of network behavior.

Basic connection features characterize communication sessions, traffic features summarize network activity over time, and content features capture suspicious behavior occurring within individual connections.

Combining these complementary feature groups enables machine learning models to identify behavioral patterns associated with cyberattacks while distinguishing them from legitimate network traffic.

3.8 Additional Features That Could Improve Performance

Although the selected features provide strong predictive performance, additional information could further improve intrusion detection.

Potential improvements include:

- Network flow statistics collected over longer observation windows.
- Packet payload characteristics.
- User behavioral profiles.
- Graph-based communication features describing relationships between hosts.
- Temporal features extracted from sequential network traffic.

Furthermore, evaluating the methodology on more recent intrusion detection datasets such as **UNSW-NB15**, **CIC-IDS2017**, or **CSE-CIC-**

IDS2018 would improve the assessment of the models under modern network conditions.

4. Reproducibility Analysis

4.1 Introduction

Reproducibility is a fundamental principle of scientific research and machine learning. A reproducible project allows other researchers to execute the same implementation, follow the same experimental procedure, and obtain comparable results using the same data and methodology.

This project was designed to ensure that every stage of the experimental pipeline is transparent and reproducible.

4.2 Code Reproducibility

The entire implementation was developed in a single Jupyter Notebook, which contains all stages of the machine learning pipeline.

The notebook includes:

- data loading,
- exploratory data analysis,
- preprocessing,
- feature engineering,
- model training,
- hyperparameter optimization,
- model evaluation,
- visualization,
- performance comparison.

Since all implementation steps are explicitly documented, another researcher can reproduce the complete workflow without requiring additional source code.

4.3 Dataset and Required Dependencies

The experiments were performed using the publicly available **NSL-KDD** dataset, which is widely used for benchmarking intrusion detection systems.

The implementation relies only on commonly used Python libraries, including:

- NumPy
- Pandas
- Matplotlib
- Seaborn
- Scikit-learn
- TensorFlow / Keras

These libraries are open-source and can be installed using standard package managers such as **pip** or **conda**. Consequently, reproducing the experimental environment is straightforward.

4.4 Hidden Preprocessing Steps

No hidden preprocessing steps were used during the implementation.

All preprocessing operations are explicitly implemented in the notebook and executed sequentially before model training.

These preprocessing steps include:

- loading the dataset,
- inspecting missing values,
- encoding categorical attributes,
- feature standardization,
- binary label transformation,
- train-test preparation,
- hyperparameter optimization,
- model evaluation.

Because every transformation is visible in the notebook, the preprocessing pipeline is fully transparent.

4.5 Modifications Introduced

Although this implementation closely follows the methodology proposed in the paper, several practical modifications were introduced.

First, the project focuses only on the **binary intrusion detection stage**, whereas the original paper proposes a two-stage hierarchical framework that continues with multi-class attack classification. Since the assignment focused on binary classification, only the first stage was reproduced.

Second, **Label Encoding** was used to encode categorical variables. The original paper describes the use of **LabelCount Encoding** during preprocessing, whereas Label Encoding was selected in this implementation because it integrates naturally with Scikit-learn and provides a simple, reproducible preprocessing pipeline.

Third, hyperparameter optimization was performed using **GridSearchCV** for every supervised learning model before the final evaluation. This ensures that each model is evaluated using an appropriate set of parameters.

Finally, the Autoencoder architecture was implemented according to the general methodology described in the paper. Since some implementation details, such as the exact anomaly detection threshold and complete architectural configuration, were not fully specified, these design choices were determined experimentally while preserving the original anomaly detection principle based on reconstruction error.

4.6 Overall Reproducibility

Overall, the methodology presented in the paper is highly reproducible.

The paper clearly describes the overall machine learning pipeline, the evaluated models, and the preprocessing methodology. However, several implementation details are omitted, making exact numerical reproduction difficult.

Despite these minor differences, the implementation developed in this project successfully reproduces the binary intrusion detection stage and obtains results that are consistent with the main conclusions reported by the authors.

This demonstrates that the proposed methodology is robust and can be reproduced successfully using publicly available datasets and standard machine learning libraries.

5. Experimental Results

5.1 Experimental Setup

The experiments were conducted using the preprocessed **NSL-KDD** dataset after completing all preprocessing and feature engineering steps described in the previous sections.

The binary intrusion detection task was evaluated using seven different machine learning models:

- Decision Tree
- Random Forest
- Linear Support Vector Machine (LinearSVC)
- AdaBoost
- Gradient Boosting
- Multi-Layer Perceptron (MLP)
- Autoencoder

To ensure a fair comparison, all supervised learning models were trained using the same training dataset and evaluated on the same testing dataset.

Hyperparameter optimization was performed using **GridSearchCV** before the final evaluation.

Unlike the supervised models, the Autoencoder was trained exclusively on normal network traffic. During testing, anomalies were detected using the reconstruction error produced by the network, following the anomaly detection principle described in the paper.

5.2 Evaluation Metrics

The performance of every model was evaluated using six complementary metrics:

- Accuracy

- Precision
- Recall
- F1-score
- ROC-AUC
- Matthews Correlation Coefficient (MCC)

Using multiple evaluation metrics provides a more comprehensive assessment than relying only on Accuracy.

For intrusion detection systems, Recall is particularly important because it measures the proportion of attacks that are correctly detected, while MCC provides a balanced evaluation even when class distributions are not perfectly balanced.

ROC-AUC evaluates the ability of each classifier to distinguish between normal and malicious traffic across different decision thresholds, making it an important complementary metric.

5.3 Experimental Results

Table 1. Performance comparison of the evaluated machine learning models.

	Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC	MCC
0	Decision Tree	0.7802	0.9654	0.6366	0.7673	0.8046	0.6202
1	Random Forest	0.7707	0.9662	0.6188	0.7544	0.9620	0.6072
2	Linear SVM	0.7512	0.9252	0.6126	0.7371	0.8702	0.5590
3	AdaBoost	0.7904	0.9637	0.6565	0.7810	0.8119	0.6340
4	Gradient Boosting	0.7767	0.9658	0.6301	0.7627	0.8003	0.6156
5	MLP	0.7956	0.9731	0.6592	0.7859	0.8175	0.6461
6	Autoencoder	0.8258	0.9304	0.7501	0.8306	0.9313	0.6717

5.4 Results Analysis

The experimental results demonstrate clear differences in performance among the evaluated machine learning models.

The **Autoencoder** achieved the best overall performance, obtaining the highest Accuracy, Recall, F1-score, and MCC. These results demonstrate the effectiveness of anomaly detection based on reconstruction error for distinguishing between normal and malicious network traffic.

Among the supervised learning models, the **Multi-Layer Perceptron (MLP)** achieved the strongest overall performance, producing the highest Precision and the highest MCC among the supervised approaches.

The **Random Forest** classifier achieved the highest ROC-AUC score, indicating excellent discrimination capability across different classification thresholds.

Although the **Linear Support Vector Machine** achieved relatively high Precision, its Recall and F1-score were lower than those of the remaining supervised learning models, making it the weakest performer in this implementation.

Overall, the obtained results support the main conclusion presented in the paper, namely that deep learning-based anomaly detection provides superior performance for binary intrusion detection compared with classical supervised learning algorithms.

5.5 Comparison with the Paper

Although the implementation follows the same binary intrusion detection methodology proposed in the paper, several differences exist between the reported results.

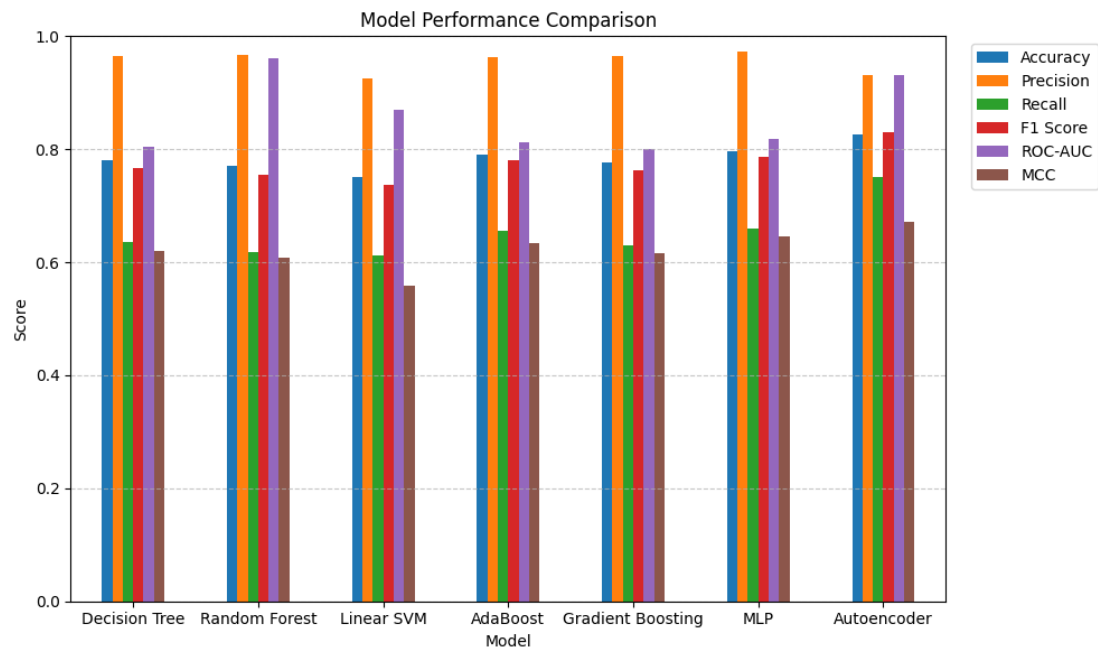
The paper reports that the Autoencoder achieved the highest overall Accuracy and F1-score among all evaluated models, while SVM and AdaBoost were the strongest supervised learning algorithms.

In this implementation, the Autoencoder also achieved the best overall performance, confirming the main conclusion of the paper. However, the **Multi-Layer Perceptron (MLP)** outperformed the remaining supervised learning models, while **LinearSVC** produced the lowest overall performance.

These differences are expected because the implementation uses independently optimized hyperparameters, a different SVM implementation (LinearSVC), and minor preprocessing differences. Nevertheless, the overall experimental trends remain consistent with the findings reported in the paper.

5.6 Performance Comparison

Figure 1. Performance comparison of the evaluated machine learning models using Accuracy, Precision, Recall, F1-score, ROC-AUC, and MCC.



6. Conclusions

6.1 Key Findings

This project successfully implemented and evaluated seven machine learning models for binary network intrusion detection using the NSL-KDD dataset.

The evaluated models included Decision Tree, Random Forest, Linear Support Vector Machine (LinearSVC), AdaBoost, Gradient Boosting, Multi-Layer Perceptron (MLP), and an Autoencoder-based anomaly detection model.

The experimental results demonstrated that all evaluated models could distinguish between normal and malicious network traffic. However, their performance varied across the different evaluation metrics.

Among all evaluated approaches, the **Autoencoder** achieved the strongest overall performance. It obtained the highest Accuracy, Recall, F1-score, and Matthews Correlation Coefficient (MCC), demonstrating

that reconstruction-based anomaly detection is highly effective for binary intrusion detection.

Among the supervised learning algorithms, the **Multi-Layer Perceptron (MLP)** achieved the best overall performance, while **Random Forest** produced the highest ROC-AUC score.

Overall, the obtained results support the main conclusion presented in the selected paper that deep learning-based anomaly detection is a highly effective approach for detecting malicious network traffic.

6.2 Lessons Learned

This project provided valuable practical experience in applying machine learning techniques to a real cybersecurity problem.

One of the most important lessons learned is that data preprocessing has a significant influence on the final performance of machine learning models. Operations such as categorical feature encoding, feature standardization, and binary label transformation are essential for producing reliable classification results.

Another important observation is that evaluating intrusion detection systems requires multiple complementary evaluation metrics rather than relying solely on Accuracy. Metrics such as Recall, F1-score, ROC-AUC, and MCC provide a much more realistic assessment of model performance, particularly in cybersecurity applications where missing an attack may have serious consequences.

The project also demonstrated that different machine learning algorithms have different strengths. While supervised learning models perform well when sufficient labeled data are available, anomaly detection methods such as the Autoencoder provide an additional advantage by learning the characteristics of normal traffic and identifying abnormal behavior through reconstruction error.

6.3 Strengths and Weaknesses

The proposed methodology offers several important strengths.

Using the same preprocessing pipeline for every evaluated model ensures a fair comparison between different machine learning approaches.

Hyperparameter optimization using GridSearchCV further improves

model performance by selecting suitable parameter combinations before evaluation.

Another strength is the inclusion of both classical supervised learning algorithms and an Autoencoder-based anomaly detection model, allowing a comprehensive comparison between supervised and unsupervised learning approaches.

Despite these advantages, several limitations remain.

The experiments were conducted exclusively on the NSL-KDD dataset, which, although widely used for research purposes, does not fully represent modern network environments or current cyberattack techniques.

Finally, several implementation details described in the paper were not fully specified, requiring some experimental decisions during implementation. These differences may explain minor discrepancies between the reported results and those obtained in this project.

6.4 Future Improvements

Several improvements could be explored in future work.

First, the complete hierarchical framework proposed in the paper could be implemented by extending the project with the second-stage multi-class attack classification.

Second, the methodology could be evaluated on more recent intrusion detection datasets such as **UNSW-NB15**, **CIC-IDS2017**, or **CSE-CIC-IDS2018** to assess its generalization capability under modern network conditions.

Future research could also investigate more advanced deep learning architectures, including Variational Autoencoders (VAEs), Graph Neural Networks (GNNs), Transformer-based models, or ensemble learning techniques for intrusion detection.

Finally, additional evaluation criteria such as inference time, computational complexity, memory consumption, and real-time deployment performance could be incorporated to provide a more comprehensive assessment of practical applicability.

References

1. Tauscher, Z., Jiang, Y., Zhang, K., Wang, J., & Song, H. (2021).
Learning to Detect: A Data-Driven Approach for Network Intrusion Detection.